

12.4.3 共享变量

我们说一个变量 *v* 是共享的，当且仅当它的一个实例被一个以上的线程引用。例如，示例程序中的变量 *cnt* 就是共享的，因为它只有一个运行时实例，并且这个实例被两个对等线程引用。在另一方面，*myid* 不是共享的，因为它的两个实例中每一个都只被一个线程引用。然而，认识到像 *msgs* 这样的本地自动变量也能被共享是很重要的。

 练习题 12.6

- A. 利用 12.4 节中的分析，为图 12-15 中的示例程序在下表的每个条目中填写“是”或者“否”。在第一列中，符号 *v.t* 表示变量 *v* 的一个实例，它驻留在线程 *t* 的本地栈中，其中 *t* 要么是 *m*(主线程)，要么是 *p0*(对等线程 0)或者 *p1*(对等线程 1)。

变量实例	主线程引用的?	对等线程0引用的?	对等线程1引用的?
<i>ptr</i>			
<i>cnt</i>			
<i>i.m</i>			
<i>msgs.m</i>			
<i>myid.p0</i>			
<i>myid.p1</i>			

- B. 根据 A 部分的分析，变量 *ptr*、*cnt*、*i*、*msgs* 和 *myid* 哪些是共享的？

12.5 用信号量同步线程

共享变量是十分方便，但是它们也引入了同步错误(synchronization error)的可能性。考虑图 12-16 中的程序 *badcnt.c*，它创建了两个线程，每个线程都对共享计数变量 *cnt* 加 1。

code/conc/badcnt.c

```
1  /* WARNING: This code is buggy! */
2  #include "csapp.h"
3
4  void *thread(void *vargp); /* Thread routine prototype */
5
6  /* Global shared variable */
7  volatile long cnt = 0; /* Counter */
8
9  int main(int argc, char **argv)
10 {
11     long niters;
12     pthread_t tid1, tid2;
13
14     /* Check input argument */
15     if (argc != 2) {
16         printf("usage: %s <niters>\n", argv[0]);
17         exit(0);
18     }
19     niters = atoi(argv[1]);
20
21     /* Create threads and wait for them to finish */
22     Pthread_create(&tid1, NULL, thread, &niters);
```

图 12-16 *badcnt.c*：一个同步不正确的计数器程序

```

23     Pthread_create(&tid2, NULL, thread, &niters);
24     Pthread_join(tid1, NULL);
25     Pthread_join(tid2, NULL);
26
27     /* Check result */
28     if (cnt != (2 * niters))
29         printf("BOOM! cnt=%ld\n", cnt);
30     else
31         printf("OK cnt=%ld\n", cnt);
32     exit(0);
33 }
34
35 /* Thread routine */
36 void *thread(void *vargp)
37 {
38     long i, niters = *((long *)vargp);
39
40     for (i = 0; i < niters; i++)
41         cnt++;
42
43     return NULL;
44 }

```

code/conc/badcnt.c

图 12-16 (续)

因为每个线程都对计数器增加了 niters 次，我们预计它的最终值是 $2 \times \text{niters}$ 。这看上去简单而直接。然而，当在 Linux 系统上运行 badcnt.c 时，我们不仅得到错误的答案，而且每次得到的答案都还不相同！

```

linux> ./badcnt 1000000
BOOM! cnt=1445085

```

```

linux> ./badcnt 1000000
BOOM! cnt=1915220

```

```

linux> ./badcnt 1000000
BOOM! cnt=1404746

```

那么哪里出错了呢？为了清晰地理解这个问题，我们需要研究计数器循环(第 40~41 行)的汇编代码，如图 12-17 所示。我们发现，将线程 i 的循环代码分解成五个部分是很有帮助的：

- H_i ：在循环头部的指令块。
- L_i ：加载共享变量 cnt 到累加寄存器 $\%rdx_i$ 的指令，这里 $\%rdx_i$ 表示线程 i 中的寄存器 $\%rdx$ 的值。
- U_i ：更新(增加) $\%rdx_i$ 的指令。
- S_i ：将 $\%rdx_i$ 的更新值存回到共享变量 cnt 的指令。
- T_i ：循环尾部的指令块。

注意头和尾只操作本地栈变量，而 L_i 、 U_i 和 S_i 操作共享计数器变量的内容。

当 badcnt.c 中的两个对等线程在一个单处理器上并发运行时，机器指令以某种顺序一个接一个地完成。因此，每个并发执行定义了两个线程中的指令的某种全序(或者交叉)。不幸的是，这些顺序中的一些将会产生正确结果，但是其他的则不会。

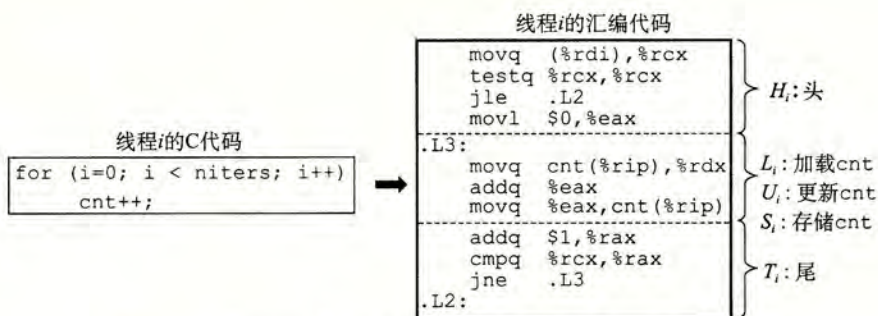


图 12-17 badcnt.c 中计数器循环(第 40~41 行)的汇编代码

这里有个关键点：一般而言，你没有办法预测操作系统是否将为你的线程选择一个正确的顺序。例如，图 12-18a 展示了一个正确的指令顺序的分步操作。在每个线程更新了共享变量 `cnt` 之后，它在内存中的值就是 2，这正是期望的值。

另一方面，图 12-18b 的顺序产生一个不正确的 cnt 的值。会发生这样的问题是因为，线程 2 在第 5 步加载 cnt，是在第 2 步线程 1 加载 cnt 之后，而在第 6 步线程 1 存储它的更新值之前。因此，每个线程最终都会存储一个值为 1 的更新后的计数器值。我们能够借助于一种叫做进度图(progress graph)的方法来阐明这些正确的和不正确的指令顺序的概念，这个图我们将在下一节中介绍。

步骤	线程	指令	%rdx ₁	%rdx ₂	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	1	U_1	1	—	0
4	1	S_1	1	—	1
5	2	H_2	—	—	1
6	2	L_2	—	1	1
7	2	U_2	—	2	1
8	2	S_2	—	2	2
9	2	T_2	—	2	2
10	1	T_1	1	—	2

a) 正确的顺序

步骤	线程	指令	%rdx ₁	%rdx ₂	cnt
1	1	H_1	—	—	0
2	1	L_1	0	—	0
3	1	U_1	1	—	0
4	2	H_2	—	—	0
5	2	L_2	—	0	0
6	1	S_1	1	—	1
7	1	T_1	1	—	1
8	2	U_2	—	1	1
9	2	S_2	—	1	1
10	2	T_2	—	1	1

b) 不正确的顺序

图 12-18 badcnt.c 中第一次循环迭代的指令顺序

 练习题 12.7 根据 badcnt.c 的指令顺序完成下表:

步骤	线程	指令	$\%rdx_1$	$\%rdx_2$	cnt
1	1	H_1	—	—	0
2	1	L_1			
3	2	H_2			
4	2	L_2			
5	2	U_2			
6	2	S_2			
7	1	U_1			
8	1	S_1			
9	1	T_1			
10	2	T_2			

这种顺序会产生一个正确的 cnt 值吗?

12.5.1 进度图

进度图(progress graph)将 n 个并发线程的执行模型化为一条 n 维笛卡儿空间中的轨迹线。每条轴 k 对应于线程 k 的进度。每个点 $(I_1, I_2, \dots, I_n)$ 代表线程 k ($k=1, \dots, n$) 已经完成了指令 I_k 这一状态。图的原点对应于没有任何线程完成一条指令的初始状态。

图 12-19 展示了 `badcnt.c` 程序第一次循环迭代的二维进度图。水平轴对应于线程 1，垂直轴对应于线程 2。点 (L_1, S_2) 对应于线程 1 完成了 L_1 而线程 2 完成了 S_2 的状态。

进度图将指令执行模型化为从一种状态到另一种状态的转换(transition)。转换被表示为一条从一点到相邻点的有向边。合法的转换是向右(线程 1 中的一条指令完成)或者向上(线程 2 中的一条指令完成)的。两条指令不能在同一时刻完成——对角线转换是不允许的。程序决不会反向运行，所以向下或者向左移动的转换也是不合法的。

一个程序的执行历史被模型化为状态空间中的一条轨迹线。图 12-20 展示了下面指令顺序对应的轨迹线：

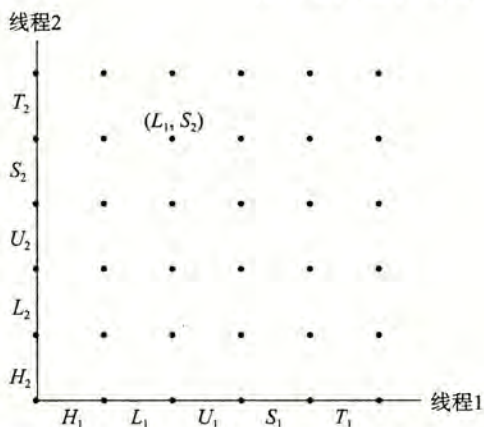
$$H_1, L_1, U_1, H_2, L_2, S_1, T_1, U_2, S_2, T_2$$


图 12-19 `badcnt.c` 第一次循环迭代的进度图

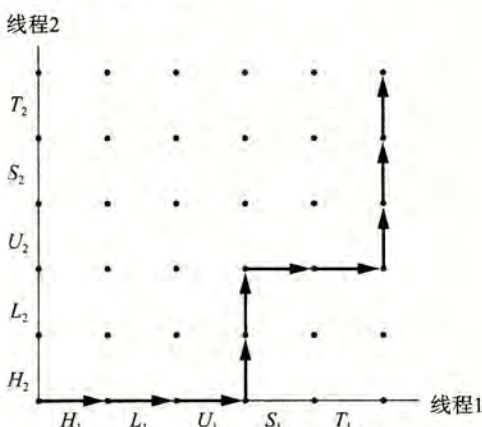


图 12-20 一个轨迹线示例

对于线程 i ，操作共享变量 `cnt` 内容的指令 (L_i, U_i, S_i) 构成了一个(关于共享变量 `cnt` 的)临界区(critical section)，这个临界区不应该和其他进程的临界区交替执行。换句话说，我们想要确保每个线程在执行它的临界区中的指令时，拥有对共享变量的互斥的访问(mutually exclusive access)。通常这种现象称为互斥(mutual exclusion)。

在进度图中，两个临界区的交集形成的状态空间区域称为不安全区(unsafe region)。图 12-21 展示了变量 `cnt` 的不安全区。注意，不安全区和与它交界的状态相毗邻，但并不包括这些状态。例如，状态 (H_1, H_2) 和 (S_1, U_2) 毗邻不安全区，但是它们并不是不安全区的一部分。绕开不安全区的轨迹线叫做安全轨迹线(safe trajectory)。相反，接触到任何不安全区的轨迹线就叫做不安全轨迹线(unsafe trajectory)。图 12-21 给出了示例程序 `badcnt.c` 的状态空间中的安全和不安全轨迹线。上面的轨迹线绕开了不安全区域的左边和上边，所以是安全的。下面的轨迹线穿越不安全区，因此是不安全的。

任何安全轨迹线都将正确地更新共享计数器。为了保证线程化程序示例的正确执行(实际上任何共享全局数据结构的并发程序的正确执行)我们必须以某种方式同步线程，使它们总是有一条安全轨迹线。一个经典的方法是基于信号量的思想，接下来我们就介绍它。

 **练习题 12.8** 使用图 12-21 中的进度图，将下列轨迹线划分为安全的或者不安全的。

A. $H_1, L_1, U_1, S_1, H_2, L_2, U_2, S_2, T_2, T_1$

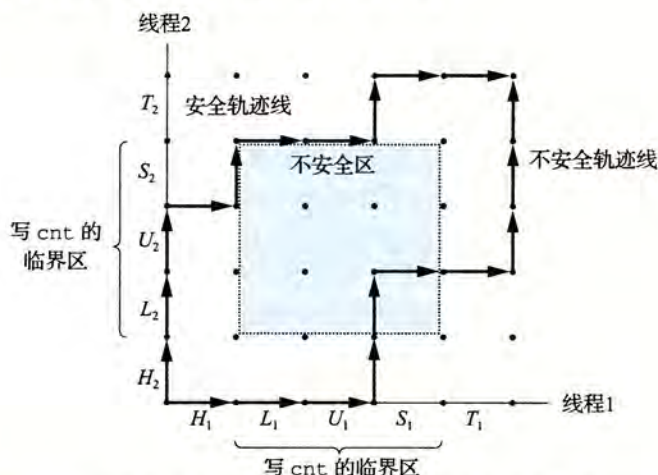
B. $H_2, L_2, H_1, L_1, U_1, S_1, T_1, U_2, S_2, T_2$ C. $H_1, H_2, L_2, U_2, S_2, L_1, U_1, S_1, T_1, T_2$ 

图 12-21 安全和不安全轨迹线。临界区的交集形成了不安全区。
绕开不安全区的轨迹线能够正确更新计数器变量

12.5.2 信号量

Edsger Dijkstra, 并发编程领域的先锋人物, 提出了一种经典的解决同步不同执行线程问题的方法, 这种方法是基于一种叫做信号量(semaphore)的特殊类型变量的。信号量 s 是具有非负整数值的全局变量, 只能由两种特殊的操作来处理, 这两种操作称为 P 和 V :

- $P(s)$: 如果 s 是非零的, 那么 P 将 s 减 1, 并且立即返回。如果 s 为零, 那么就挂起这个线程, 直到 s 变为非零, 而一个 V 操作会重启这个线程。在重启之后, P 操作将 s 减 1, 并将控制返回给调用者。
- $V(s)$: V 操作将 s 加 1。如果有任何线程阻塞在 P 操作等待 s 变成非零, 那么 V 操作会重启这些线程中的一个, 然后该线程将 s 减 1, 完成它的 P 操作。

P 中的测试和减 1 操作是不可分割的, 也就是说, 一旦预测信号量 s 变为非零, 就会将 s 减 1, 不能有中断。 V 中的加 1 操作也是不可分割的, 也就是加载、加 1 和存储信号量的过程中没有中断。注意, V 的定义中没有定义等待线程被重启的顺序。唯一的要求是 V 必须只能重启一个正在等待的线程。因此, 当有多个线程在等待同一个信号量时, 你不能预测 V 操作要重启哪一个线程。

P 和 V 的定义确保了一个正在运行的程序绝不可能进入这样一种状态, 也就是一个正确初始化了的信号量有一个负值。这个属性称为信号量不变性(semaphore invariant), 为控制并发程序的轨迹线提供了强有力的工具, 在下一节中我们将看到。

Posix 标准定义了许多操作信号量的函数。

```
#include <semaphore.h>

int sem_init(sem_t *sem, 0, unsigned int value);
int sem_wait(sem_t *s);    /* P(s) */
int sem_post(sem_t *s);    /* V(s) */
```

返回: 若成功则为 0, 若出错则为 -1。

`sem_init` 函数将信号量 `sem` 初始化为 `value`。每个信号量在使用前必须初始化。针对我们的目的，中间的参数总是零。程序分别通过调用 `sem_wait` 和 `sem_post` 函数来执行 *P* 和 *V* 操作。为了简明，我们更喜欢使用下面这些等价的 *P* 和 *V* 的包装函数：

```
#include "csapp.h"

void P(sem_t *s); /* Wrapper function for sem_wait */
void V(sem_t *s); /* Wrapper function for sem_post */
```

返回：无。

旁注 *P* 和 *V* 名字的起源

Edsger Dijkstra(1930—2002)出生于荷兰。名字 *P* 和 *V* 来源于荷兰语单词 *Proberen* (测试)和 *Verhogen*(增加)。

12.5.3 使用信号量来实现互斥

信号量提供了一种很方便的方法来确保对共享变量的互斥访问。基本思想是将每个共享变量(或者一组相关的共享变量)与一个信号量 *s*(初始为 1)联系起来，然后用 *P(s)* 和 *V(s)* 操作将相应的临界区包围起来。

以这种方式来保护共享变量的信号量叫做二元信号量(binary semaphore)，因为它的值总是 0 或者 1。以提供互斥为目的的二元信号量常常也称为互斥锁(mutex)。在一个互斥锁上执行 *P* 操作称为对互斥锁加锁。类似地，执行 *V* 操作称为对互斥锁解锁。对一个互斥锁加了锁但是还没有解锁的线程称为占用这个互斥锁。一个被用作一组可用资源的计数器的信号量被称为计数信号量。

图 12-22 中的进度图展示了我们如何利用二元信号量来正确地同步计数器程序示例。每个状态都标出了该状态中信号量 *s* 的值。关键思想是这种 *P* 和 *V* 操作的结合创建了一组

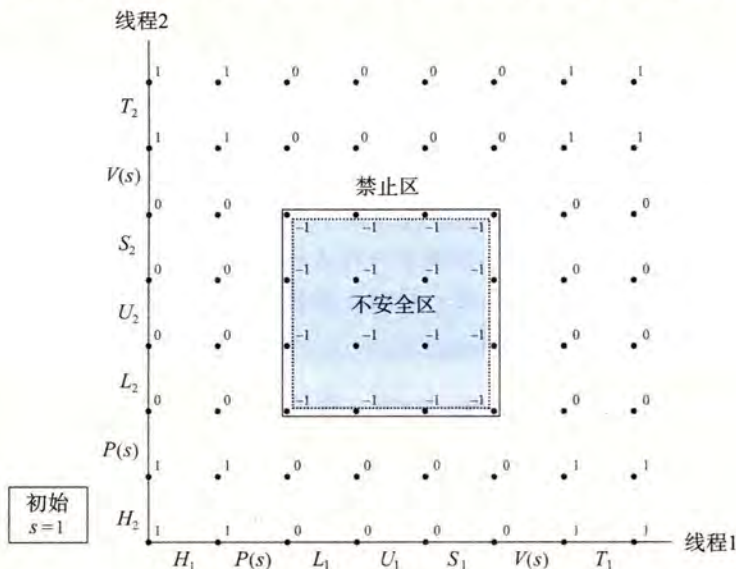


图 12-22 使用信号量来互斥。 $s < 0$ 的不可行状态定义了一个禁止区，禁止区完全包括了不安全区，阻止了实际可行的轨迹线接触到不安全区

状态，叫做禁止区(forbidden region)，其中 $s < 0$ 。因为信号量的不变性，没有实际可行的轨迹线能够包含禁止区中的状态。而且，因为禁止区完全包括了不安全区，所以没有实际可行的轨迹线能够接触不安全区的任何部分。因此，每条实际可行的轨迹线都是安全的，而且不管运行时指令顺序是怎样的，程序都会正确地增加计数器值。

从可操作的意义上来说，由 P 和 V 操作创建的禁止区使得在任何时间点上，在被包围的临界区中，不可能有多个线程在执行指令。换句话说，信号量操作确保了对临界区的互斥访问。

总的来说，为了用信号量正确同步图 12-16 中的计数器程序示例，我们首先声明一个信号量 mutex：

```
volatile long cnt = 0; /* Counter */
sem_t mutex;          /* Semaphore that protects counter */
```

然后在主例程中将 mutex 初始化为 1：

```
Sem_init(&mutex, 0, 1); /* mutex = 1 */
```

最后，我们通过把在线程例程中对共享变量 cnt 的更新包围 P 和 V 操作，从而保护它们：

```
for (i = 0; i < niters; i++) {
    P(&mutex);
    cnt++;
    V(&mutex);
}
```

当我们运行这个正确同步的程序时，现在它每次都能产生正确的结果了。

```
linux> ./goodcnt 1000000
OK cnt=2000000
```

```
linux> ./goodcnt 1000000
OK cnt=2000000
```

旁注 进度图的局限性

进度图给了我们一种较好的方法，将在单处理器上的并发程序执行可视化，也帮助我们理解为什么需要同步。然而，它们确实也有局限性，特别是对于在多处理器上的并发执行，在多处理器上一组 CPU/高速缓存对共享同一个主存。多处理器的工作方式是进度图不能解释的。特别是，一个多处理器内存系统可以处于一种状态，不对应于进度图中任何轨迹线。不管如何，结论总是一样的：无论是在单处理器还是多处理器上运行程序，都要同步你对共享变量的访问。

12.5.4 利用信号量来调度共享资源

除了提供互斥之外，信号量的另一个重要作用是调度对共享资源的访问。在这种场景中，一个线程用信号量操作来通知另一个线程，程序状态中的某个条件已经为真了。两个经典而有用的例子是生产者-消费者和读者-写者问题。

1. 生产者-消费者问题

图 12-23 给出了生产者-消费者问题。生产者和消费者线程共享一个有 n 个槽的有限缓冲区。生产者线程反复地生成新的项目(item)，并把它们插入到缓冲区中。消费者线程不断地

从缓冲区中取出这些项目，然后消费(使用)它们。也可能有多个生产者和消费者的变种。

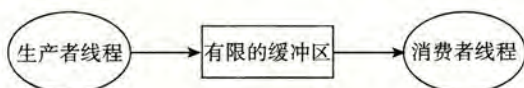


图 12-23 生产者-消费者问题。生产者产生项目并把它们插入到一个有限的缓冲区中。消费者从缓冲区中取出这些项目，然后消费它们

因为插入和取出项目都涉及更新共享变量，所以我们必须保证对缓冲区的访问是互斥的。但是只保证互斥访问是不够的，我们还需要调度对缓冲区的访问。如果缓冲区是满的(没有空的槽位)，那么生产者必须等待直到有一个槽位变为可用。与之相似，如果缓冲区是空的(没有可取用的项目)，那么消费者必须等待直到有一个项目变为可用。

生产者-消费者的相互作用在现实系统中是很普遍的。例如，在一个多媒体系统中，生产者编码视频帧，而消费者解码并在屏幕上呈现出来。缓冲区的目的是为了减少视频流的抖动，而这种抖动是由各个帧的编码和解码时与数据相关的差异引起的。缓冲区为生产者提供了一个槽位池，而为消费者提供一个已编码的帧池。另一个常见的示例是图形用户接口设计。生产者检测到鼠标和键盘事件，并将它们插入到缓冲区中。消费者以某种基于优先级的方式从缓冲区取出这些事件，并显示在屏幕上。

在本节中，我们将开发一个简单的包，叫做 SBUF，用来构造生产者-消费者程序。在下一节里，我们会看到如何用它来构造一个基于预线程化(prethreading)的有趣的并发服务器。SBUF 操作类型为 `sbuf_t` 的有限缓冲区(图 12-24)。项目存放在一个动态分配的 n 项整数数组(buf)中。`front` 和 `rear` 索引值记录该数组中的第一项和最后一项。三个信号量同步对缓冲区的访问。`mutex` 信号量提供互斥的缓冲区访问。`slots` 和 `items` 信号量分别记录空槽位和可用项目的数量。

```

code/conc/sbuf.h
1  typedef struct {
2      int *buf;           /* Buffer array */
3      int n;             /* Maximum number of slots */
4      int front;         /* buf[(front+1)%n] is first item */
5      int rear;          /* buf[rear%n] is last item */
6      sem_t mutex;       /* Protects accesses to buf */
7      sem_t slots;       /* Counts available slots */
8      sem_t items;       /* Counts available items */
9  } sbuf_t;

code/conc/sbuf.h
  
```

图 12-24 `sbuf_t`: SBUF 包使用的有限缓冲区

图 12-25 给出了 SBUF 函数的实现。`sbuf_init` 函数为缓冲区分配堆内存，设置 `front` 和 `rear` 表示一个空的缓冲区，并为三个信号量赋初始值。这个函数在调用其他三个函数中的任何一个之前调用一次。`sbuf_deinit` 函数是当应用程序使用完缓冲区时，释放缓冲区存储的。`sbuf_insert` 函数等待一个可用的槽位，对互斥锁加锁，添加项目，对互斥锁解锁，然后宣布有一个新项目可用。`sbuf_remove` 函数是与 `sbuf_insert` 函数对称的。在等待一个可用的缓冲区项目之后，对互斥锁加锁，从缓冲区的前面取出该项目，对互斥锁解锁，然后发信号通知一个新的槽位可供使用。

code/conc/sbuf.c

```

1  #include "csapp.h"
2  #include "sbuf.h"
3
4  /* Create an empty, bounded, shared FIFO buffer with n slots */
5  void sbuf_init(sbuf_t *sp, int n)
6  {
7      sp->buf = Calloc(n, sizeof(int));
8      sp->n = n;
9      sp->front = sp->rear = 0;
10     Sem_init(&sp->mutex, 0, 1);
11     Sem_init(&sp->slots, 0, n);
12     Sem_init(&sp->items, 0, 0);
13 }
14
15 /* Clean up buffer sp */
16 void sbuf_deinit(sbuf_t *sp)
17 {
18     Free(sp->buf);
19 }
20
21 /* Insert item onto the rear of shared buffer sp */
22 void sbuf_insert(sbuf_t *sp, int item)
23 {
24     P(&sp->slots);
25     P(&sp->mutex);
26     sp->buf[(++sp->rear)%(sp->n)] = item;
27     V(&sp->mutex);
28     V(&sp->items);
29 }
30
31 /* Remove and return the first item from buffer sp */
32 int sbuf_remove(sbuf_t *sp)
33 {
34     int item;
35     P(&sp->items);
36     P(&sp->mutex);
37     item = sp->buf[(++sp->front)%(sp->n)];
38     V(&sp->mutex);
39     V(&sp->slots);
40     return item;
41 }

```

code/conc/sbuf.c

图 12-25 SBUF: 同步对有限缓冲区并发访问的包



练习题 12.9 设 p 表示生产者数量, c 表示消费者数量, 而 n 表示以项目单元为单位的缓冲区大小。对于下面的每个场景, 指出 `sbuf_insert` 和 `sbuf_remove` 中的互斥锁信号量是否是必需的。

- A. $p=1, c=1, n>1$
- B. $p=1, c=1, n=1$
- C. $p>1, c>1, n=1$

2. 读者-写者问题

读者-写者问题是互斥问题的一个概括。一组并发的线程要访问一个共享对象, 例如

一个主存中的数据结构，或者一个磁盘上的数据库。有些线程只读对象，而其他的线程只修改对象。修改对象的线程叫做写者。只读对象的线程叫做读者。写者必须拥有对对象的独占的访问，而读者可以和无限多个其他的读者共享对象。一般来说，有无限多个并发的读者和写者。

读者-写者交互在现实系统中很常见。例如，一个在线航空预定系统中，允许有无限多个客户同时查看座位分配，但是正在预订座位的客户必须拥有对数据库的独占的访问。再来看另一个例子，在一个多线程缓存 Web 代理中，无限多个线程可以从共享页面缓存中取出已有的页面，但是任何向缓存中写入一个新页面的线程必须拥有独占的访问。

读者-写者问题有几个变种，分别基于读者和写者的优先级。第一类读者-写者问题，读者优先，要求不要让读者等待，除非已经把使用对象的权限赋予了一个写者。换句话说，读者不会因为有一个写者在等待而等待。第二类读者-写者问题，写者优先，要求一旦一个写者准备好可以写，它就会尽可能快地完成它的写操作。同第一类问题不同，在一个写者后到达的读者必须等待，即使这个写者也是在等待。

图 12-26 给出了一个对第一类读者-写者问题的解答。同许多同步问题的解答一样，这个解答很微妙，极具欺骗性地简单。信号量 w 控制对访问共享对象的临界区的访问。信号量 mutex 保护对共享变量 readcnt 的访问， readcnt 统计当前在临界区中的读者数量。每当一个写者进入临界区时，它对互斥锁 w 加锁，每当它离开临界区时，对 w 解锁。这就保证了任意时刻临界区中最多只有一个写者。另一方面，只有第一个进入临界区的读者对 w 加锁，而只有最后一个离开临界区的读者对 w 解锁。当一个读者进入和离开临界区时，如果还有其他读者在临界区中，那么这个读者会忽略互斥锁 w 。这就意味着只要还有一个读者占用互斥锁 w ，无限多数量的读者可以没有障碍地进入临界区。

对这两种读者-写者问题的正确解答可能导致饥饿 (starvation)，饥饿就是一个线程无限期地阻塞，无法进展。例如，图 12-26 所示的解答中，如果有读者不断地到达，写者就可能无限期地等待。

```
/* Global variables */
int readcnt;    /* Initially = 0 */
sem_t mutex, w; /* Both initially = 1 */

void reader(void)
{
    while (1) {
        P(&mutex);
        readcnt++;
        if (readcnt == 1) /* First in */
            P(&w);
        V(&mutex);

        /* Critical section */
        /* Reading happens */

        P(&mutex);
        readcnt--;
        if (readcnt == 0) /* Last out */
            V(&w);
        V(&mutex);
    }
}

void writer(void)
{
    while (1) {
        P(&w);

        /* Critical section */
        /* Writing happens */

        V(&w);
    }
}
```

图 12-26 对第一类读者-写者问题的解答。
读者优先级高于写者

 **练习题 12.10** 图 12-26 所示的对第一类读者-写者问题的解答给予读者较高的优先级,但是从某种意义上说,这种优先级是很弱的,因为一个离开临界区的写者可能重启一个在等待的写者,而不是一个在等待的读者。描述出一个场景,其中这种弱优先级会导致一群写者使得一个读者饥饿。

旁注 其他同步机制

我们已经向你展示了如何利用信号量来同步线程,主要是因为它们简单、经典,并且有一个清晰的语义模型。但是你应该知道还是存在着其他同步技术的。例如,Java 线程是用一种叫做 Java 监控器(Java Monitor)[48]的机制来同步的,它提供了对信号量互斥和调度能力的更高级别的抽象;实际上,监控器可以用信号量来实现。再来看一个例子,Pthreads 接口定义了一组对互斥锁和条件变量的同步操作。Pthreads 互斥锁被用来实现互斥。条件变量用来调度对共享资源的访问,例如在一个生产者-消费者程序中的有限缓冲区。

12.5.5 综合:基于预线程化的并发服务器

我们已经知道了如何使用信号量来访问共享变量和调度对共享资源的访问。为了帮助你更清晰地理解这些思想,让我们把它们应用到一个基于称为预线程化(prethreading)技术的并发服务器上。

在图 12-14 所示的并发服务器中,我们为每一个新客户端创建了一个新线程。这种方法的缺点是我们为每一个新客户端创建一个新线程,导致不小的代价。一个基于预线程化的服务器试图通过使用如图 12-27 所示的生产者-消费者模型来降低这种开销。服务器是由一个主线程和一组工作者线程构成的。主线程不断地接受来自客户端的连接请求,并将得到的连接描述符放在一个有限缓冲区中。每一个工作者线程反复地从共享缓冲区中取出描述符,为客户端服务,然后等待下一个描述符。

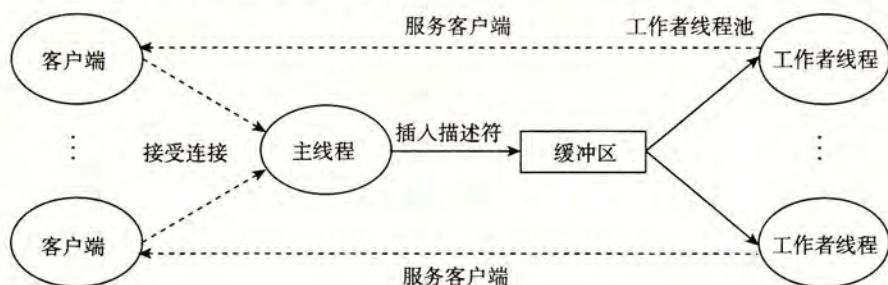


图 12-27 预线程化的并发服务器的组织结构。一组现有的线程不断地取出和处理来自有限缓冲区的已连接描述符

图 12-28 显示了我们怎样用 SBUF 包来实现一个预线程化的并发 echo 服务器。在初始化了缓冲区 sbuf (第 24 行)后,主线程创建了一组工作者线程(第 25~26 行)。然后它进入了无限的服务器循环,接受连接请求,并将得到的已连接描述符插入到缓冲区 sbuf 中。每个工作者线程的行为都非常简单。它等待直到它能从缓冲区中取出一个已连接描述符(第 39 行),然后调用 echo_cnt 函数回送客户端的输入。

图 12-29 所示的函数 echo_cnt 是图 11-22 中的 echo 函数的一个版本,它在全局变量 byte_cnt 中记录了从所有客户端接收到的累计字节数。这是一段值得研究的有趣代码,因为它向你展示了一个从线程例程调用的初始化程序包的一般技术。在这种情况下,我们

需要初始化 `byte_cnt` 计数器和 `mutex` 信号量。一个方法是我们为 SBUF 和 RIO 程序包使用过的，它要求主线程显式地调用一个初始化函数。另外一个方法，在此显示的，是当第一次有某个线程调用 `echo_cnt` 函数时，使用 `pthread_once` 函数(第 19 行)去调用初始化函数。这个方法的优点是它使程序包的使用更加容易。这种方法的缺点是每一次调用 `echo_cnt` 都会导致调用 `pthread_once` 函数，而在大多数时候它没有做什么有用的事。

code/conc/echoservt-pre.c

```

1  #include "csapp.h"
2  #include "sbuf.h"
3  #define NTHREADS 4
4  #define SBUFSIZE 16
5
6  void echo_cnt(int connfd);
7  void *thread(void *vargp);
8
9  sbuf_t sbuf; /* Shared buffer of connected descriptors */
10
11 int main(int argc, char **argv)
12 {
13     int i, listenfd, connfd;
14     socklen_t clientlen;
15     struct sockaddr_storage clientaddr;
16     pthread_t tid;
17
18     if (argc != 2) {
19         fprintf(stderr, "usage: %s <port>\n", argv[0]);
20         exit(0);
21     }
22     listenfd = Open_listenfd(argv[1]);
23
24     sbuf_init(&sbuf, SBUFSIZE);
25     for (i = 0; i < NTHREADS; i++) /* Create worker threads */
26         Pthread_create(&tid, NULL, thread, NULL);
27
28     while (1) {
29         clientlen = sizeof(struct sockaddr_storage);
30         connfd = Accept(listenfd, (SA *) &clientaddr, &clientlen);
31         sbuf_insert(&sbuf, connfd); /* Insert connfd in buffer */
32     }
33 }
34
35 void *thread(void *vargp)
36 {
37     Pthread_detach(pthread_self());
38     while (1) {
39         int connfd = sbuf_remove(&sbuf); /* Remove connfd from buffer */
40         echo_cnt(connfd);                /* Service client */
41         Close(connfd);
42     }
43 }

```

code/conc/echoservt-pre.c

图 12-28 一个预线程化的并发 echo 服务器。这个服务器使用的是有一个生产者和多个消费者的生产者-消费者模型


```

code/conc/echo-cnt.c

1  #include "csapp.h"
2
3  static int byte_cnt; /* Byte counter */
4  static sem_t mutex; /* and the mutex that protects it */
5
6  static void init_echo_cnt(void)
7  {
8      Sem_init(&mutex, 0, 1);
9      byte_cnt = 0;
10 }
11
12 void echo_cnt(int connfd)
13 {
14     int n;
15     char buf[MAXLINE];
16     rio_t rio;
17     static pthread_once_t once = PTHREAD_ONCE_INIT;
18
19     Pthread_once(&once, init_echo_cnt);
20     Rio_readinitb(&rio, connfd);
21     while((n = Rio_readlineb(&rio, buf, MAXLINE)) != 0) {
22         P(&mutex);
23         byte_cnt += n;
24         printf("server received %d (%d total) bytes on fd %d\n",
25             n, byte_cnt, connfd);
26         V(&mutex);
27         Rio_writen(connfd, buf, n);
28     }
29 }
code/conc/echo-cnt.c

```

图 12-29 echo_cnt: echo 的一个版本, 它对从客户端接收的所有字节计数

一旦程序包被初始化, echo_cnt 函数会初始化 RIO 带缓冲区的 I/O 包(第 20 行), 然后回送从客户端接收到的每一个文本行。注意, 在第 23~25 行中对共享变量 byte_cnt 的访问是被 P 和 V 操作保护的。

旁注 基于线程的事件驱动程序

I/O 多路复用不是编写事件驱动程序的唯一方法。例如, 你可能已经注意到我们刚才开发的并发的预线程化的服务器实际上是一个事件驱动服务器, 带有主线程和工作者线程的简单状态机。主线程有两种状态(“等待连接请求”和“等待可用的缓冲区槽位”)、两个 I/O 事件(“连接请求到达”和“缓冲区槽位变为可用”)和两个转换(“接受连接请求”和“插入缓冲区项目”)。类似地, 每个工作者线程有一个状态(“等待可用的缓冲项目”)、一个 I/O 事件(“缓冲区项目变为可用”)和一个转换(“取出缓冲区项目”)。

12.6 使用线程提高并行性

到目前为止, 在对并发的研究中, 我们都假设并发线程是在单处理器系统上执行的。